

RUST PROGRAMMING

Enums and Patterns

Kamal kumar mukiri, Apt Computing Labs



ENUMS AND PATTERNS



- Enums
 - Basics
 - Generic Enums
- Patterns
 - Literals, Variables, and Wildcards
 - Tuple and struct patterns
 - Array and Slice Patterns
 - Reference Patterns
 - Match Guards

Enums: Basics

```
1 use std::env;
2 #[derive(Debug)]
0 implementations
1 enum SeverityLevel {
4     Low,
5     Medium,
6     High,
7     Critical,
8 }
9
10 fn find_severity(identiy: i32) -> SeverityLevel {
11     match identiy {
12         1..=10 => SeverityLevel::Low,
13         11..=20 => SeverityLevel::Medium,
14         21..=30 => SeverityLevel::High,
15         _ => SeverityLevel::Critical,
16     }
17 }
18
19 fn main() {
20     println!("Severity Level = {:?}", find_severity(10));
21 }
```

```
● kamalmukiri@Kamals-MacBook-Air Basics % ./enums_1
Severity Level = Low
```

1. Defining enum (like C)
2. Using enum as return value

Enums: Having multiple datatypes

```
1 use std::env;
2 #[derive(Debug)]
3 0 implementations
4 enum SeverityLevel {
5     Low,
6     Medium,
7     High,
8     Critical,
9     ErrorMsg(String), 1
10 }
11 fn find_severity(identiy: i32) -> SeverityLevel {
12     match identiy {
13         1..=10 => SeverityLevel::Low,
14         11..=20 => SeverityLevel::Medium,
15         21..=30 => SeverityLevel::High,
16         30..=40 => SeverityLevel::Critical,
17         2 _ => SeverityLevel::ErrorMsg("Invalid range".to_string()) 2
18     }
19 }
20
21 fn main() {
22     println!("Severity Level = {:?}", find_severity(10));
23     println!("Severity Level = {:?}", find_severity(41));
24 }
```

1. Including String as a member of enum
2. Returning the message in case of abnormal cases

Enums: Import constructors of “enum”

```
1 use std::env;
2 #[derive(Debug)]
3 0 implementations
4 enum SeverityLevel {
5     Low,
6     Medium,
7     High,
8     Critical,
9     ErrorMsg(String),
10 }
11 use self::SeverityLevel::*; 1
12
13 fn find_severity(identiy: i32) -> SeverityLevel {
14     match identiy {
15         1..=10 => Low,
16         11..=20 => Medium, 2
17         21..=30 => High,
18         30..=40 => Critical,
19         _ => ErrorMsg("Invalid range".to_string())
20     }
21 }
22
23 fn main() {
24     println!("Severity Level = {:?}", find_severity(10));
25     println!("Severity Level = {:?}", find_severity(41));
26 }
```

```
kamalmukiri@Kamals-MacBook-Air Basics % ./enums_multiple_datatypes
Severity Level = Low
Severity Level = ErrorMsg("Invalid range")
```

1. Including enum members without scope resolution using “use self:.”
2. Utilizing members without enum name.

Note: This is not recommended as per standard.
(Recall in C++, “using namespace std;”

Enums: Challenge: Find error

```
2  #[derive(Debug)]
   0 implementations
3  enum SeverityLevel {
4      Low,
5      Medium,
6      High,
7      Critical,
8      ErrorMsg(String),
9  }
10
11  use self::SeverityLevel::*;
12
13  fn find_severity(identiy: i32) -> SeverityLevel {
14      match identiy {
15          1..=10 => Low,
16          11..=20 => Medium,
17          21..=30 => High,
18          30..=40 => Critical,
19          _ => ErrorMsg("Invalid range".to_string())
20      }
21  }
22
23  fn main() {
24      println!("Severity Level = {:?}", find_severity(10));
25      println!("Severity Level = {:?}", find_severity(41));
26
27      let LowSev: SeverityLevel = Low;
28      let typeCast: i32;
29      typeCast = LowSev as i32;
30      println!("typeCast = {:?}", typeCast);
31  }
```


Enums: Challenge: Find error

```
2  #[derive(Debug)]
   0 implementations
3  enum SeverityLevel {
4      Low,
5      Medium,
6      High,
7      Critical,
8      ErrorMsg(String),
9  }
10
11 use self::SeverityLevel::*;
12
13 fn find_severity(identiy: i32) -> SeverityLevel {
14     match identiy {
15         1..=10 => Low,
16         11..=20 => Medium,
17         21..=30 => High,
18         30..=40 => Critical,
19         _ => ErrorMsg("Invalid range".to_string())
20     }
21 }
22
23 fn main() {
24     println!("Severity Level = {:?}", find_severity(10));
25     println!("Severity Level = {:?}", find_severity(41));
26
27     let LowSev: SeverityLevel = Low;
28     let typeCast: i32;
29     typeCast = LowSev as i32;
30     println!("typeCast = {:?}", typeCast);
31 }
```

1. error[E0605]: non-primitive cast: `SeverityLevel` as `i32`

an `as` expression can be used to convert enum types to numeric types only if the enum type is unit-only or field-less

Enums: Challenge: Find error : Fixing

2 `#[derive(Debug)]`

```
3 enum SeverityLevel {  
4     Low,  
5     Medium,  
6     High,  
7     Critical,  
8     //ErrorMsg(String),  
9 }
```

1



```
10  
11 use self::SeverityLevel::*;  
12  
13 fn find_severity(identiy: i32) -> SeverityLevel {  
14     match identiy {  
15         1..=10 => Low,  
16         11..=20 => Medium,  
17         21..=30 => High,  
18         _ => Critical,  
19         //_ => ErrorMsg("Invalid range".to_string())  
20     }  
21 }  
22  
23 fn main() {  
24     println!("Severity Level = {:?}", find_severity(10));  
25     println!("Severity Level = {:?}", find_severity(41));  
26  
27     let LowSev: SeverityLevel = Low;  
28     let typeCast: i32;  
29     typeCast = LowSev as i32;  
30     println!("typeCast = {:?}", typeCast);  
31 }  
32 ✨  
33 |
```

1. Modify the enum to “unit-only”

Enums: Challenge: Find error

```
2  #[derive(Debug)]
   0 implementations
3  enum SeverityLevel {
4      Low,
5      Medium,
6      High,
7      Critical,
8      ErrorMsg(String),
9  }
10
11 use self::SeverityLevel::*;
12
13 fn find_severity(identiy: i32) -> SeverityLevel {
14     match identiy {
15         1..=10 => Low,
16         11..=20 => Medium,
17         21..=30 => High,
18         30..=40 => Critical,
19         _ => ErrorMsg("Invalid range".to_string())
20     }
21 }
22
23 fn main() {
24     println!("Severity Level = {:?}", find_severity(10));
25     println!("Severity Level = {:?}", find_severity(41));
26
27     let LowSev: SeverityLevel = Low;
28     let typeCast: i32;
29     typeCast = LowSev as i32;
30     println!("typeCast = {:?}", typeCast);
31 }
```

1. error[E0605]: non-primitive cast: `SeverityLevel` as `i32`

an `as` expression can be used to convert enum types to numeric types only if the enum type is unit-only or field-less

Enums: Import constructors of enum

```
1 use std::env;
2 #[derive(Debug)]
3 0 implementations
4 enum SeverityLevel {
5     Low,
6     Medium,
7     High,
8     Critical,
9     ErrorMsg(String),
10 }
11 use self::SeverityLevel::*; 1
12
13 fn find_severity(identiy: i32) -> SeverityLevel {
14     match identiy {
15         1..=10 => Low,
16         11..=20 => Medium, 2
17         21..=30 => High,
18         30..=40 => Critical,
19         _ => ErrorMsg("Invalid range".to_string())
20     }
21 }
22
23 fn main() {
24     println!("Severity Level = {:?}", find_severity(10));
25     println!("Severity Level = {:?}", find_severity(41));
26 }
```

```
kamalmukiri@Kamals-MacBook-Air Basics % ./enums_multiple_datatypes
Severity Level = Low
Severity Level = ErrorMsg("Invalid range")
```

1. Including enum members without scope resolution using “use self:.”
2. Utilizing members without enum name.

Note: This is not recommended as per standard.
(Recall in C++, “using namespace std;”

Patterns: Literals

```
fn main() {  
    let x = 42;  
  
    match x {  
        1 => println!("One"),  
        42 => println!("The answer to life, the  
            universe, and everything"),  
        _ => println!("Something else"),  
    }  
}
```

Literals: Literals are direct values like numbers or strings that are used in pattern matching.

Enums

Basics

Generic Enums

Patterns

Literals, Variables,
and Wildcards

Tuple and struct
patterns

Array and Slice
Patterns

Reference Patterns

Match Guards

Patterns: Variables

```
fn main() {  
    let x = 100;  
  
    match x {  
        1 => println!("One"),  
        other => println!("Got another number: {}",  
            other),  
    }  
}
```

Variables: Variables in patterns bind to values and can be used inside the match arms.

Enums

Basics

Generic Enums

Patterns

Literals, Variables,
and Wildcards

Tuple and struct
patterns

Array and Slice
Patterns

Reference Patterns

Match Guards

Patterns: Wild cards

```
fn main() {  
    let x = 77;  
  
    match x {  
        1 => println!("One"),  
        _ => println!("This is something else"),  
    }  
}
```

Wild cards: The `_` wildcard ignores any value without binding it to a variable. It's useful for cases where you don't need the specific value.

Enums

Basics

Generic Enums

Patterns

Literals, Variables,
and Wildcards

Tuple and struct
patterns

Array and Slice
Patterns

Reference Patterns

Match Guards

Patterns: Tuples

Enums

Basics

Generic Enums

Patterns

Literals, Variables,
and Wildcards

**Tuple and struct
patterns**

Array and Slice
Patterns

Reference Patterns

Match Guards

```
fn main() {  
    let point = (0, 7);  
  
    match point {  
        (0, y) => println!("Point is on the y-axis at y = {}", y),  
        (x, 0) => println!("Point is on the x-axis at x = {}", x),  
        (x, y) => println!("Point is at ({}", x, y),  
    }  
}
```

Tuples are simple groupings of values. You can destructure and match on tuples in Rust using pattern matching.

Patterns: Structure

Enums

Basics

Generic Enums

Patterns

Literals, Variables,
and Wildcards

Tuple and struct
patterns

Array and Slice
Patterns

Reference Patterns

Match Guards

```
struct Point {  
    x: i32,  
    y: i32,  
}  
  
fn func() -> Option<Point> {  
    Some(Point{x:1, y:2})  
}  
  
fn main() {  
    match func() {  
        Some(Point { x: 0, y }) => println!("On the y-axis at y = {}",  
            y),  
        Some(Point { x, y: 0 }) => println!("On the x-axis at x = {}",  
            x),  
        Some(Point { x, y }) => println!("Point is at ({}, {})", x, y),  
        None => println!("Nothing"),  
    }  
}
```

Struct patterns allow you to match and destructure struct fields. You can match by field names or destructure the entire struct.

Patterns: Arrays

Enums

Basics

Generic Enums

Patterns

Literals, Variables,
and Wildcards

Tuple and struct
patterns

Array and Slice
Patterns

Reference Patterns

Match Guards

```
fn get_list() -> [i32;3] {  
    [1, 2, 3]  
}  
  
fn main() {  
    match get_list() {  
        [1, 2, 3] => println!("Array matches exactly [1, 2, 3]!"),  
        [1, ..] => println!("Array starts with 1!"),  
        [.., 3] => println!("Array ends with 3!"),  
        _ => println!("No match!"),  
    }  
}
```

An array pattern is used to match arrays of a specific length.

Patterns: Arrays with slices

Enums

Basics

Generic Enums

Patterns

Literals, Variables,
and Wildcards

Tuple and struct
patterns

Array and Slice
Patterns

Reference Patterns

Match Guards

```
fn match_array(array:&[i32]) {  
    match array {  
        &[1, .., 3] => println!("Array matches exactly [1, 2, 3]!" ),  
        &[1,2,..] => println!("Array starts with 1!" ),  
        &[.., 3] => println!("Array ends with 3!" ),  
        _ => println!("No match!" ),  
    }  
}  
  
fn main() {  
    let array1:[i32;4] = [1,2,3,4];  
    let array2:[i32;10] = [1,2,3,4,5,6,7,8,9,0];  
    match_array (&array1);  
    match_array (&array2);  
}
```

Slice patterns are used with slices (&[T]) where the length of the collection is not known at compile-time. The syntax is similar to array patterns but operates on dynamically sized collections.

nnwarp()

What Does unwrap() Do?

- The **.unwrap()** method is defined for Option and Result types in Rust.
- For **Option**, **.unwrap()** extracts the contained value if it's Some, and panics if it's None.
- For **Result**, **.unwrap()** extracts the value if it's Ok, and panics if it's Err.

Enums

Basics

Generic Enums

Patterns

Literals, Variables,
and Wildcards

Tuple and struct
patterns

Array and Slice
Patterns

Reference Patterns

Match Guards

Defining own “unwrap()”

```
enum SpreadsheetCell {  
    Int(i32),  
    Float(f64),  
    Text(String),  
}  
  
impl SpreadsheetCell {  
    // Custom unwrap for `Int` variant  
    pub fn unwrap_int(self) -> i32 {  
        if let SpreadsheetCell::Int(value) = self {  
            value  
        } else {  
            panic!("Called `unwrap_int` on a non-Int variant!");  
        }  
    }  
  
    // Custom unwrap for `Float` variant  
    pub fn unwrap_float(self) -> f64 {  
        if let SpreadsheetCell::Float(value) = self {  
            value  
        } else {  
            panic!("Called `unwrap_float` on a non-Float variant!");  
        }  
    }  
  
    // Custom unwrap for `Text` variant  
    pub fn unwrap_text(self) -> String {  
        if let SpreadsheetCell::Text(value) = self {  
            value  
        } else {  
            panic!("Called `unwrap_text` on a non-Text variant!");  
        }  
    }  
}  
  
fn main() {  
    let cell = SpreadsheetCell::Int(42);  
  
    // Using the custom unwrap method  
    let value = cell.unwrap_int();  
    println!("The value is: {}", value);  
}
```

You can add a custom method named `unwrap_*` for each variant in your enum to extract its value and panic if it's the wrong variant.

expect()

Enums

Basics

Generic Enums

Patterns

Literals, Variables,
and Wildcards

Tuple and struct
patterns

Array and Slice
Patterns

Reference Patterns

Match Guards

The `.expect()` method is called on `Result` or `Option` types and is similar to `.unwrap()`, but with an added benefit: you can specify an error message to display if the `Result` is `Err` or the `Option` is `None`.

Defining own “unwrap()”

```
enum SpreadsheetCell {  
    Int(i32),  
    Float(f64),  
    Text(String),  
}  
  
impl SpreadsheetCell {  
    // Custom unwrap for `Int` variant  
    pub fn unwrap_int(self) -> i32 {  
        if let SpreadsheetCell::Int(value) = self {  
            value  
        } else {  
            panic!("Called `unwrap_int` on a non-Int variant!");  
        }  
    }  
  
    // Custom unwrap for `Float` variant  
    pub fn unwrap_float(self) -> f64 {  
        if let SpreadsheetCell::Float(value) = self {  
            value  
        } else {  
            panic!("Called `unwrap_float` on a non-Float variant!");  
        }  
    }  
  
    // Custom unwrap for `Text` variant  
    pub fn unwrap_text(self) -> String {  
        if let SpreadsheetCell::Text(value) = self {  
            value  
        } else {  
            panic!("Called `unwrap_text` on a non-Text variant!");  
        }  
    }  
}  
  
fn main() {  
    let cell = SpreadsheetCell::Int(42);  
  
    // Using the custom unwrap method  
    let value = cell.unwrap_int();  
    println!("The value is: {}", value);  
}
```

You can add a custom method named `unwrap_*` for each variant in your enum to extract its value and panic if it's the wrong variant.